API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use

API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  ```

  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

### 2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
# API Design Guidelines v1

## Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

## 1. Authentication Approach

### 1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

### 1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

### 1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

## 2. Rate Limits

### 2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.
  ```

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```

- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours

- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use
API Design Guidelines v1

Introduction
This document outlines the fundamental principles and best practices for designing APIs within

---

1. Authentication Approach

1.1. Authentication Method
- **Token-Based Authentication:** All APIs must implement token-based authentication using **OA
- **API Keys:** For public or less sensitive endpoints, API keys may be used but should be supp

1.2. Token Management
- Tokens must be issued via a secure authorization server.
- Tokens should have a configurable expiration time, recommended between 15 minutes to 24 hours
- Refresh tokens should be used to obtain new access tokens without re-authentication.

1.3. Security Considerations
- All API requests must be made over **HTTPS** to ensure data encryption.
- Tokens should be included in the `Authorization` header:
  ```
  Authorization: Bearer <access_token>
  ```
- Implement proper validation and scope checks on the server side.

---

2. Rate Limits

2.1. Purpose
Rate limiting prevents abuse, ensures fair usage, and maintains service availability.

2.2. Default Limits
- **Per User/IP:** 1000 requests per hour.
- **Per Application:** 10,000 requests per day.

2.3. Implementation
- Rate limits are enforced via response headers:
  ```
  X-RateLimit-Limit: <max_requests>
  X-RateLimit-Remaining: <remaining_requests>
  X-RateLimit-Reset: <timestamp>
  ```
- When limits are exceeded, respond with HTTP status code **429 Too Many Requests**.

2.4. Custom Limits
- Higher rate limits may be granted upon request and approval, based on use